

Sistemas Distribuidos: Entrega 3

Vicente Jorquera Gamonal
Profesor: Nicolas Hidalgo
Sección 2

21 de Noviembre, 2025

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	4
3. Arquitectura y Metodología	5
3.1. Diseño de la Infraestructura	5
3.2. Flujo de Datos y Metodología	5
3.3. Datos Utilizados	6
4. Resultados del Análisis Lingüístico	7
4.1. Métricas Generales de los Datasets	7
4.2. Top Palabras: Respuestas Humanas	7
4.3. Top Palabras: Respuestas IA (LLM)	8
4.4. Visualización Comparativa	8
5. Discusión Comparativa	9
5.1. Formalidad y Estructura vs. Coloquialismo	9
5.2. Objetividad vs. Subjetividad	9
5.3. La Intersección: “Can” y “Like”	9
6. Discusión Técnica y Arquitectónico	10
6.1. Justificación de Apache Pig	10
6.2. Procesamiento Batch (Hadoop) vs. Streaming (Flink)	10
7. Conclusión	11

1. Introducción

En las etapas previas del proyecto, el desarrollo se centró en la construcción de una arquitectura orientada a eventos para el procesamiento de solicitudes en tiempo real, utilizando tecnologías como Apache Kafka y Spark Streaming. Si bien este enfoque permitió gestionar el flujo inmediato de datos, los sistemas distribuidos modernos requieren también de capacidades robustas para el procesamiento de datos históricos (*Batch Processing*) con el fin de extraer valor a largo plazo.

El presente informe documenta la tercera entrega del proyecto, cuyo foco se desplaza hacia el análisis lingüístico *offline*. El objetivo central es explotar el repositorio de datos acumulados para identificar patrones de vocabulario y comparar las estructuras léxicas entre respuestas generadas por humanos y aquellas sintetizadas por Inteligencia Artificial (LLM).

Para este análisis, se utilizan dos fuentes de datos heterogéneas en volumen: un dataset masivo de respuestas históricas provenientes de *Yahoo! Answers* y un conjunto de respuestas sintéticas generadas específicamente para esta etapa mediante la API del modelo **Gemini**. Cabe mencionar que, debido a incidencias técnicas en el entorno de desarrollo original y a las limitaciones de cuota de la API durante la fase de recuperación, el dataset del LLM (158 registros) presenta una cardinalidad significativamente menor al dataset humano. Sin embargo, esta disparidad dimensional no obstaculiza el objetivo principal de la tarea: validar la capacidad de la arquitectura Hadoop y la lógica de los scripts de Pig para procesar, limpiar y analizar texto de manera distribuida, independientemente del volumen de entrada.

Para materializar este servicio, se implementa un clúster de **Apache Hadoop** containerizado con Docker, utilizando HDFS para el almacenamiento distribuido y **Apache Pig** para la abstracción de tareas MapReduce, permitiendo realizar tokenización, limpieza de *stopwords* y conteo de frecuencia de manera eficiente y escalable.

2. Objetivos

2.1. Objetivo General

Diseñar, implementar y desplegar un servicio de análisis *offline* (Batch) basado en el ecosistema Hadoop, capaz de procesar conjuntos de datos de respuestas humanas y sintéticas para extraer estadísticas lingüísticas y patrones de vocabulario comparativos, utilizando Apache Pig como motor de procesamiento.

2.2. Objetivos Específicos

Para dar cumplimiento al objetivo general, se establecen los siguientes objetivos específicos:

- **Implementación de Infraestructura Distribuida:** Desplegar un clúster Hadoop funcional mediante contenedores Docker, orquestando los servicios de *NameNode*, *DataNode* y *ResourceManager* a través de `docker-compose`, asegurando la persistencia y accesibilidad de HDFS.
- **Ingesta y Gestión de Datos:** Establecer un flujo de trabajo para la carga de datos no estructurados (`.txt`) desde el sistema de archivos local hacia el sistema de archivos distribuido (HDFS), garantizando la disponibilidad de los datasets de *Yahoo! Answers* y del LLM (Gemini).
- **Desarrollo de Lógica MapReduce con Pig:** Codificar scripts en lenguaje *Pig Latin* que implementen el pipeline de procesamiento de texto requerido: conversión a minúsculas, eliminación de signos de puntuación, tokenización y filtrado de palabras vacías (*stopwords*).
- **Validación Funcional del Procesamiento:** Ejecutar el análisis de frecuencia de palabras (*WordCount*) sobre ambos datasets, demostrando la capacidad del sistema para manejar tanto grandes volúmenes de datos (Humanos) como muestras reducidas (LLM) bajo la misma lógica de procesamiento.
- **Análisis Comparativo:** Evaluar los resultados obtenidos para identificar divergencias en el uso del lenguaje entre humanos e IA, normalizando las métricas de frecuencia para permitir una comparación cualitativa válida a pesar de la diferencia en el tamaño de las muestras.

3. Arquitectura y Metodología

Para abordar el desafío del análisis lingüístico *offline*, se diseñó una arquitectura basada en el paradigma de computación distribuida, utilizando contenedores para encapsular los servicios del ecosistema Hadoop. A diferencia de la entrega anterior, centrada en la baja latencia (Streaming), esta arquitectura prioriza el rendimiento (*Throughput*) y la escalabilidad para el procesamiento por lotes.

3.1. Diseño de la Infraestructura

La infraestructura se desplegó sobre un sistema operativo **Ubuntu Linux ejecutándose de manera nativa** (*bare metal*), lo que elimina la sobrecarga (*overhead*) asociada a la virtualización tradicional y maximiza el acceso a los recursos de hardware (CPU/RAM) para los contenedores.

El clúster de Hadoop fue orquestado mediante **docker-compose**, garantizando la portabilidad del entorno. Se compone de los siguientes nodos interconectados en una red virtual dedicada:

- **NameNode:** Nodo maestro del sistema de archivos HDFS. Gestiona los metadatos y la ubicación de los bloques de datos.
- **DataNode:** Nodo esclavo encargado del almacenamiento físico de los datos. En este entorno, se configuró una instancia para validar la replicación y persistencia.
- **ResourceManager & NodeManager (YARN):** Responsables de la gestión de recursos del clúster y de la programación de las tareas de MapReduce.
- **Pig Client:** Contenedor cliente con *Apache Pig 0.17.0*, actuando como interfaz para la inyección de scripts y comandos HDFS.

3.2. Flujo de Datos y Metodología

La metodología de implementación siguió un flujo de trabajo ETL (*Extract, Transform, Load*) adaptado al entorno Big Data:

1. Preparación e Ingesta de Datos:

- **Fuentes:** Se prepararon dos datasets principales (Humanos y LLM).
- **Recursos Auxiliares:** Para el filtrado de ruido, se generó el archivo `stopwords.txt` utilizando el modelo de lenguaje **Gemini**. Esto permitió crear una lista exhaustiva y actualizada de conectores, preposiciones y artículos irrelevantes para el análisis semántico, optimizando la calidad de la limpieza.
- **Carga (Staging):** Los archivos se transfirieron desde el sistema de archivos local (Ubuntu) hacia HDFS bajo el directorio `/input` mediante comandos `hdfs dfs -put`.

2. **Procesamiento con Pig:** Se utilizó **Apache Pig** como capa de abstracción para evitar la complejidad de Java MapReduce. El script `analisis.pig` implementa la siguiente lógica de transformación:

- **Normalización:** Conversión de texto a minúsculas (**LOWER**).
- **Limpieza (Regex):** Eliminación de puntuación y caracteres especiales mediante expresiones regulares (**REPLACE**), conservando solo caracteres alfanuméricos.
- **Tokenización:** Descomposición de líneas en palabras individuales (**TOKENIZE**).
- **Filtrado:** Aplicación de un *Replicated Join* entre los tokens y la lista de stopwords generada por IA para descartar términos vacíos.
- **Agregación:** Conteo de frecuencias (*WordCount*) y ordenamiento descendente.

3. **Salida:** Los resultados procesados se almacenaron en HDFS y posteriormente se consolidaron en archivos CSV locales para su análisis comparativo.

3.3. Datos Utilizados

El análisis se realizó sobre dos conjuntos de datos heterogéneos:

- **Dataset Humano:** Volcado histórico de *Yahoo! Answers* (~460 MB), representando lenguaje natural no estructurado.
- **Dataset Sintético (LLM):** Muestra de 158 respuestas (~64 KB) regeneradas mediante la API de Gemini tras incidencias técnicas con el dataset original. A pesar de la diferencia de volumen, esta muestra es suficiente para validar la lógica del pipeline y contrastar el vocabulario formal de la IA frente al lenguaje humano.

4. Resultados del Análisis Lingüístico

En esta sección se presentan las estadísticas descriptivas obtenidas tras la ejecución del proceso MapReduce sobre ambos conjuntos de datos. Los resultados reflejan el vocabulario más frecuente utilizado en cada contexto, normalizado a través del proceso de limpieza y eliminación de *stopwords*.

4.1. Métricas Generales de los Datasets

Es fundamental notar la diferencia de magnitud entre las fuentes para interpretar correctamente las frecuencias absolutas.

Métrica	Dataset Humanos (Yahoo!)	Dataset LLM (Gemini)
Total de Palabras (Tokens)	46,745,378	5,942
Tamaño en Disco	~463 MB	64 KB
Naturaleza	Coloquial / Ruidoso	Formal / Estructurado

Tabla 1: Comparación dimensional de los datasets procesados.

4.2. Top Palabras: Respuestas Humanas

El análisis del dataset masivo de Yahoo! Answers revela un predominio de verbos modales, términos de acción y expresiones de opinión, consistentes con la naturaleza de un foro de discusión.

Rank	Palabra	Frecuencia	% del Total
1	can	464,256	0.99 %
2	will	387,089	0.83 %
3	one	264,394	0.57 %
4	like	253,496	0.54 %
5	get	253,351	0.54 %
6	just	249,038	0.53 %
7	don	196,427	0.42 %
8	people	194,525	0.42 %
9	good	177,901	0.38 %
10	com	177,613	0.38 %

Tabla 2: Top 10 palabras más frecuentes en respuestas humanas.

Observación: Destaca la presencia de “com” (posición 10) y “www” (posición 16 en los datos crudos), evidenciando que los usuarios humanos frecuentemente comparten enlaces externos como parte de sus respuestas, un patrón de “ruido” que sobrevivió a la limpieza alfanumérica básica.

4.3. Top Palabras: Respuestas IA (LLM)

Las respuestas generadas por Gemini muestran un patrón léxico radicalmente diferente, caracterizado por un lenguaje más técnico y explicativo.

Rank	Palabra	Frecuencia	% del Total
1	often	26	0.44 %
2	use	24	0.40 %
3	like	23	0.39 %
4	specific	18	0.30 %
5	yes	18	0.30 %
6	high	18	0.30 %
7	can	18	0.30 %
8	highly	16	0.27 %
9	primarily	15	0.25 %
10	based	14	0.24 %

Tabla 3: Top 10 palabras más frecuentes en respuestas del LLM.

4.4. Visualización Comparativa

Para facilitar la comparación directa entre ambos perfiles léxicos, la Figura 1 presenta gráficamente la distribución de frecuencia de los 20 términos más utilizados en cada dataset.

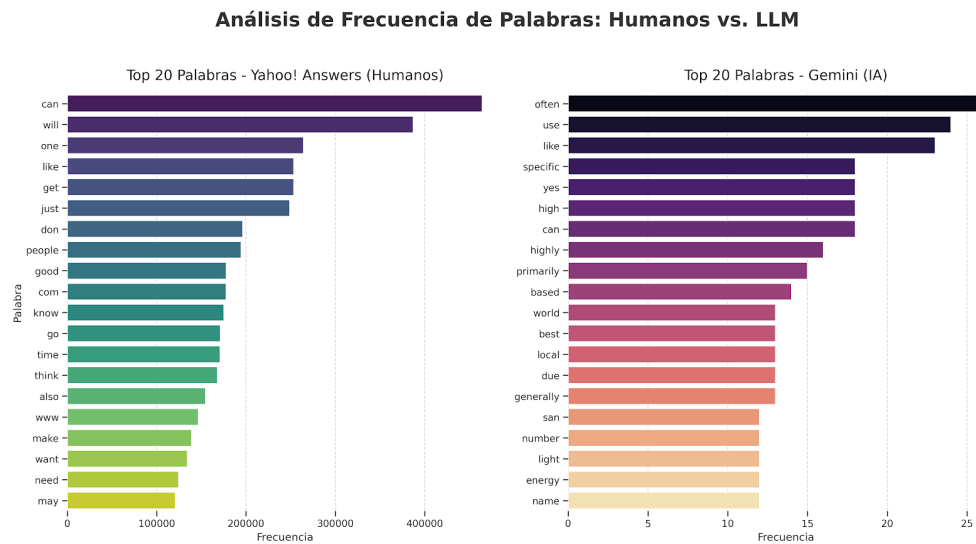


Figura 1: Comparación de las palabras más frecuentes: El vocabulario humano (izquierda) muestra una alta concentración en verbos comunes, mientras que el vocabulario de la IA (derecha) se distribuye en términos más técnicos y adverbios.

5. Discusión Comparativa

Al contrastar ambos listados, emergen tres patrones distintivos que diferencian la comunicación humana de la sintética:

5.1. Formalidad y Estructura vs. Coloquialismo

El vocabulario del LLM es notablemente más **formal y expositivo**. Términos como *often*, *specific*, *highly*, *primarily*, *based* y *generally* (Top 20) sugieren que la IA construye sus respuestas estructurando argumentos, matizando afirmaciones y citando bases lógicas.

En contraste, el vocabulario humano es **operativo y subjetivo**. Palabras como *get*, *just*, *want*, *think*, *know* y *good* reflejan opiniones personales, deseos o instrucciones directas. La aparición de *don* (probablemente residuo de "don't" tras eliminar el apóstrofe) refuerza el tono conversacional informal.

5.2. Objetividad vs. Subjetividad

Mientras que los humanos utilizan frecuentemente verbos cognitivos o volitivos (*think*, *want*, *need*), la IA carece de estos términos en su cima de frecuencia. En su lugar, utiliza adverbios de frecuencia y modo (*often*, *generally*, *highly*), lo que indica un esfuerzo algorítmico por proveer información generalizable y neutra en lugar de opiniones personales.

5.3. La Intersección: "Can" y "Like"

Las palabras *can* y *like* son las únicas intersecciones significativas en el Top 10 de ambos grupos, pero su uso pragmático podría diferir:

- **Can:** En humanos (Rank 1) sugiere posibilidad o permiso ("Can I do this?"). En la IA, suele denotar capacidad técnica ("This method can be used...").
- **Like:** En humanos (Rank 4) a menudo expresa preferencia ("I like") o actúa como muletilla informal. En la IA (Rank 3), es probable que se utilice para comparaciones ("It functions like...").

6. Discusión Técnica y Arquitectónico

6.1. Justificación de Apache Pig

La elección de Apache Pig resultó crucial para la productividad del desarrollo. Implementar la lógica de *WordCount* con limpieza de texto y *joins* (para stopwords) en Java MapReduce nativo habría requerido escribir múltiples clases (Mapper, Reducer, Driver) y manejar manualmente la serialización de tipos. Pig permitió condensar esta lógica en menos de 15 líneas de código declarativo (**FOREACH**, **FILTER**, **GROUP**), abstrayendo la complejidad del grafo de ejecución MapReduce subyacente.

6.2. Procesamiento Batch (Hadoop) vs. Streaming (Flink)

Basado en la experiencia acumulada entre la Tarea 2 y 3, se evidencia que:

- **Streaming (Flink/Spark):** Es ideal para eventos “uno a uno” donde la latencia es crítica (ej. responder a un usuario en tiempo real). Sin embargo, realizar un conteo global de vocabulario en streaming es ineficiente, ya que requeriría mantener un estado (state) gigante en memoria que crece indefinidamente.
- **Batch (Hadoop):** Es el modelo correcto para este problema. Al tener todos los datos históricos disponibles (“bounded data”), Hadoop puede optimizar el procesamiento dividiendo el archivo de 460MB en bloques HDFS y procesándolos en paralelo, sin la presión de responder en milisegundos. La alta latencia de inicio de los jobs de Hadoop (segundos o minutos) es irrelevante para un análisis analítico que se ejecuta una sola vez al día o al mes.

7. Conclusión

La implementación de esta tercera etapa completó el ciclo de vida de los datos del proyecto. Se demostró que una arquitectura de Big Data clásica (Hadoop + Pig) sigue siendo la herramienta estándar para el análisis profundo de grandes volúmenes de texto.

Desde la perspectiva lingüística, se concluye que, aunque los LLMs actuales (como Gemini) son capaces de generar respuestas coherentes, su huella léxica es claramente distinguible de la humana: la IA es un redactor técnico, preciso y formal, mientras que el usuario humano de Yahoo! Answers es coloquial, directo y propenso al ruido. Esta distinción es vital para futuros sistemas que busquen detectar contenido generado automáticamente o humanizar las respuestas de los bots.

Finalmente, todo el código desarrollado para este proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/Jorquerinh0/Tarea3SD.git>